Week 8: ISA & Machine Language, Addressing Modes

Midterm Exam Preparation
Week 8

According to your syllabus, Week 8 covers two major parts:

1. Instruction Set Architecture (ISA) & Machine Language
    - Instruction Set Architecture
    - Machine language
    - Assembly language
    - Opcode and operands
    - Instruction format
    - Instruction categories
2. Addressing Modes
    - Immediate addressing
    - Register addressing
    - Direct addressing
    - Register indirect addressing
    - Based addressing
    - Indexed addressing
    - Based-indexed addressing

Let's understand all topics step by step, with examples and important exam questions.

Part 1: ISA & Machine Language

1. What is Instruction Set Architecture (ISA)?

Definition: Instruction Set Architecture (ISA) is the interface between hardware and software that defines the instructions a processor can understand and execute.

In simple words, ISA defines what a processor can do and how software communicates with the processor.

It specifies:

- Instructions supported by the processor.
- Types of registers.
- Data types and sizes.
- Addressing modes.
- Instruction formats.

- Memory access rules.

Example

Suppose a processor supports these instructions:

```
MOV AX, BX
ADD AX, BX
SUB AX, BX
MUL BX
```

These instructions are part of the processor's instruction set.

The ISA defines what these instructions mean and how the processor executes them.

Why is ISA important?

1. It provides a standard interface between software and hardware.
2. It defines the instructions a CPU can execute.
3. It helps programmers write software for a processor.
4. It defines how data and memory are accessed.

Remember: ISA defines the instructions and rules that a processor supports.

2. What is Machine Language?

Definition: Machine language is the lowest-level programming language that consists of binary instructions (0s and 1s) directly understood by the CPU.

A processor executes machine-language instructions directly.

Example

An instruction is represented internally as binary bits:

```
10110000 00000101
```

These bits represent an instruction in a particular machine's instruction set.

Characteristics of Machine Language

- Consists of binary digits (0 and 1).
- Directly understood by the CPU.
- Does not require translation into another programming language before execution.
- Difficult for humans to read and write.
- Machine-specific.

Remember: Machine language = Binary language understood by the CPU.

3. What is Assembly Language?

Definition: Assembly language is a low-level programming language that uses symbolic instructions, called mnemonics, to represent machine-language operations.

Instead of writing binary instructions, programmers use readable instruction names.

Example

Machine language:

```
10110000 00000101
```

Assembly language:

```
MOV AL, 05H
```

Here:

- MOV is a mnemonic.
- AL is a register.
- 05H is a hexadecimal value.

Characteristics of Assembly Language

1. Uses mnemonics such as MOV, ADD, SUB, and MUL.
2. Easier to understand than machine language.
3. Requires an assembler to convert it into machine code.
4. Is processor-specific.
5. Provides direct control over registers and memory.

What is an Assembler?

Definition: An assembler is a program that converts assembly-language instructions into machine code.

Example:

```
Assembly Language
       |
       v
    Assembler
       |
       v
 Machine Language
```

```
  |
  v
CPU
```

4. Difference Between Machine Language and Assembly Language

Important Exam Question

Machine Language	Assembly Language
Uses binary digits (0 and 1)	Uses mnemonics
Directly understood by CPU	Requires an assembler
Difficult to read and write	Easier to read and write
No assembler is needed to translate it	Assembler converts it into machine code
Example: Binary instruction	Example: MOV AX, BX

Exam answer: Machine language consists of binary instructions directly executed by the CPU, whereas assembly language uses symbolic instructions and mnemonics that are translated into machine code by an assembler.

5. What is an Opcode?

Definition: An opcode (Operation Code) is the part of an instruction that specifies which operation the CPU must perform.

It tells the processor what action is required.

Examples:

Opcode	Operation
MOV	Transfer data
ADD	Addition
SUB	Subtraction
MUL	Multiplication
DIV	Division
AND	Logical AND
OR	Logical OR
JMP	Jump to another location
CMP	Compare values

Example:

```
ADD AX, BX
```

Here:

- Opcode = ADD
- Operation = Addition

The CPU understands that it must perform an addition operation.

Remember: Opcode tells the CPU WHAT to do.

6. What are Operands?

Definition: Operands are the data, registers, or memory locations on which an instruction performs an operation.

They specify the values or locations involved in an operation.

Example:

```
ADD AX, BX
```

Here:

- Opcode = ADD
- Operand 1 = AX
- Operand 2 = BX

The CPU adds the contents of AX and BX.

Another example:

```
MOV AX, 10
```

Here:

- Opcode = MOV
- Operand 1 = AX
- Operand 2 = 10

The value 10 is moved into AX.

Difference Between Opcode and Operand

Opcode	Operand

Specifies the operation	Specifies the data or location
Tells what to do	Tells on what to do
Example: ADD	Example: AX, BX

Easy trick:

- Opcode = What operation?
- Operand = Which data?

7. Instruction Format

Definition: Instruction format is the arrangement or structure of bits within a machine-language instruction.

It defines how an instruction is divided into fields.

A typical instruction contains:

1. Opcode field
2. Operand field(s)
3. Addressing-mode information (when required)

[Java] JVM and CPU compatibility. Let’s find out why JVM and CPU must be… | by John Park | Medium
medium.com

Example

Consider:

```
ADD AX, BX
```

Conceptually, the instruction contains:

Field	Meaning
Opcode	ADD
Destination operand	AX
Source operand	BX

The actual machine-code encoding depends on the processor's ISA.

Common Instruction Formats

A. Zero-Address Instruction

An instruction has no explicit operand addresses.

Example:

```
ADD
```

This format is commonly associated with stack-based machines, where operands are taken from the stack.

B. One-Address Instruction

An instruction contains one explicit operand. Another operand is often implied.

Example:

```
ADD X
```

A common accumulator-based interpretation is:

```
AC = AC + X
```

C. Two-Address Instruction

An instruction contains two explicit operands.

Example:

```
ADD AX, BX
```

The first operand is generally the destination and also one of the source operands:

```
AX = AX + BX
```

D. Three-Address Instruction

An instruction contains three explicit operands.

Example:

```
ADD R1, R2, R3
```

Meaning:

```
R1 = R2 + R3
```

These are general instruction-format concepts. Actual operand formats depend on the processor architecture.

Comparison

Format	Number of Explicit Addresses	Example
Zero-address	0	ADD
One-address	1	ADD X
Two-address	2	ADD AX, BX
Three-address	3	ADD R1, R2, R3

8. Instruction Categories

Definition: Instruction categories are groups of instructions classified according to the operations they perform.

The main categories include:

A. Data Transfer Instructions

These instructions transfer data between registers, memory, and other locations.

Examples:

```
MOV AX, BX
MOV AX, 10
PUSH AX
POP BX
XCHG AX, BX
```

Common instructions:

Instruction	Purpose
MOV	Copies data
PUSH	Pushes data onto stack
POP	Removes data from stack
XCHG	Exchanges values
IN	Reads data from an input port
OUT	Sends data to an output port

Example:

```
MOV AX, 10
```

The value 10 is copied into AX.

B. Arithmetic Instructions

These instructions perform mathematical operations.

Examples:

```
ADD AX, BX
SUB AX, BX
MUL BX
DIV BX
INC AX
DEC BX
```

Instruction	Purpose
ADD	Addition
SUB	Subtraction
MUL	Multiplication
DIV	Division
INC	Increment by 1
DEC	Decrement by 1

Example:

```
MOV AX, 10
ADD AX, 5
```

Result:

```
AX = 15
```

C. Logical Instructions

These instructions perform bitwise logical operations.

Examples:

```
AND AX, BX
OR AX, BX
XOR AX, BX
NOT AX
```

Instruction	Purpose
AND	Logical AND
OR	Logical OR
XOR	Exclusive OR

NOT	Inverts bits
TEST	Performs logical AND for flag testing without storing the result

Example:

```
  1010
AND 1100
--------
  1000
```

D. Control Transfer Instructions

These instructions change the normal flow of program execution.

Examples:

```
JMP 2000H
JE LABEL
JNE LABEL
CALL FUNCTION
RET
```

Instruction	Purpose
JMP	Unconditional jump
JE	Jump if equal
JNE	Jump if not equal
CALL	Calls a procedure
RET	Returns from a procedure
LOOP	Repeats instructions using CX

E. Shift and Rotate Instructions

These instructions shift or rotate bits.

Examples:

```
SHL AX, 1
SHR AX, 1
ROL AX, 1
ROR AX, 1
```

Instruction	Purpose
SHL	Shifts bits left
SHR	Shifts bits right
ROL	Rotates bits left
ROR	Rotates bits right

Example:

```
Original: 00001000
SHL 1:    00010000
```

F. String Instructions

These instructions process strings or sequences of bytes and words.

Examples:

- MOVSB – Move a byte from source to destination.
- MOVSW – Move a word from source to destination.
- CMPSB – Compare bytes.
- LODSB – Load a byte into AL.
- STOSB – Store AL into destination memory.

Part 2: Addressing Modes

9. What are Addressing Modes?

Very Important Long Question

Definition: Addressing modes are the methods used by a processor to specify where an instruction's operand is located or how to obtain it.

In simple words, addressing modes tell the CPU where to find the data required by an instruction.

Why are Addressing Modes Needed?

1. To access data stored in registers.
2. To use constant values directly.
3. To access data in memory.
4. To access arrays and tables.
5. To support flexible memory operations.

Your syllabus includes seven addressing modes. Let's understand each one.

10. Immediate Addressing Mode

Definition: In immediate addressing mode, the actual data value is provided directly inside the instruction.

The operand is a constant value.

Example

```
MOV AX, 1234H
```

Here:

- Destination = AX
- Immediate value = 1234H

The CPU directly loads 1234H into AX.

```
AX = 1234H
```

Another example:

```
MOV CL, 05H
```

Now:

```
CL = 05H
```

Important points:

- Data is provided directly in the instruction.
- No separate memory lookup is needed to obtain the immediate value.
- Commonly used to load constants.

Remember: Immediate = Data is directly given.

PPT - Chapter four 80x86 Instruction Set （2） 8086 Addressing Modes PowerPoint Presentation - ID:3196488
slideserve.com

11. Register Addressing Mode

Definition: In register addressing mode, the operand is stored in a CPU register.

The instruction specifies which register contains the data.

Example

```
MOV AX, BX
```

Suppose:

```
BX = 100
```

After execution:

```
AX = 100
```

Another example:

```
ADD AX, BX
```

Suppose:

```
AX = 10
BX = 20
```

The result becomes:

```
AX = 30
```

Important points:

- Operand is located in a register.
- Very fast because registers are inside the CPU.
- Does not require a separate memory data access for the operand.

Remember: Register = Data is inside a CPU register.

PPT - Chapter four 80x86 Instruction Set （2） 8086 Addressing Modes PowerPoint Presentation - ID:3196488
slideserve.com

12. Direct Addressing Mode

Definition: In direct addressing mode, the instruction directly specifies the memory offset address where the operand is located.

In 8086, the default segment for a simple direct memory operand is generally DS.

Example

```
MOV AX, [2000H]
```

Here:

- 2000H is the direct memory offset.
- The CPU accesses the data stored at DS:2000H.
- The data is loaded into AX.

Suppose:

```
Memory[DS:2000H] = 1234H
```

After execution:

```
AX = 1234H
```

Important: The brackets indicate memory access.

Compare:

```
MOV AX, 2000H
```

This loads the immediate value 2000H into AX.

But:

```
MOV AX, [2000H]
```

This loads the value stored in memory at offset 2000H into AX.

These two instructions are different.

PPT - Chapter four 80x86 Instruction Set （2） 8086 Addressing Modes PowerPoint Presentation - ID:3196488
slideserve.com

Remember: Direct = Memory address is directly provided.

13. Register Indirect Addressing Mode

Definition: In register indirect addressing mode, a register contains the memory offset address of the operand.

Instead of specifying the memory address directly in the instruction, the instruction uses a register that holds the address.

In 8086, the valid registers for this type of memory addressing are:

- BX
- BP
- SI
- DI

Example

```
MOV BX, 2000H
MOV AX, [BX]
```

Suppose:

```
BX = 2000H
Memory[DS:2000H] = 1234H
```

The instruction:

```
MOV AX, [BX]
```

loads the data stored at the memory location pointed to by BX.

Result:

```
AX = 1234H
```

Important points:

- The register contains the memory offset.
- The operand is in memory, not inside the register.
- BX, BP, SI, and DI are used.
- BX, SI, and DI normally use DS by default; BP normally uses SS.

PPT - EECE 374: Microprocessor Architecture and Applications Chapter 3 PowerPoint Presentation - ID:3824592
slideserve.com

Remember: Register indirect = Register contains the address, memory contains the data.

14. Based Addressing Mode

Definition: In based addressing mode, the effective memory offset is calculated by adding a displacement to a base register.

The base registers used in 8086 are:

- BX
- BP

Formula

\[\boxed{EA=Base\ Register+Displacement} \]

EA means Effective Address.

Example

```
MOV BX, 1000H
MOV AX, [BX + 20H]
```

Given:

```
BX = 1000H
Displacement = 0020H
```

Step 1: Calculate effective address.

\[EA=1000H+0020H \]
\[EA=1020H \]

Step 2: CPU accesses memory at DS:1020H.

Suppose:

```
Memory[DS:1020H] = 1234H
```

Then:

```
AX = 1234H
```

Another example

```
MOV BP, 2000H
MOV AX, [BP + 10H]
```
\[EA=2000H+0010H \]
\[EA=2010H \]

Because BP is used, the default segment is SS.

Remember: Based = Base register + displacement.

15. Indexed Addressing Mode

Definition: In indexed addressing mode, the effective memory offset is calculated by adding an index register to an optional displacement.

The index registers used are:

- SI – Source Index
- DI – Destination Index

Formula

\[\boxed{EA=Index\ Register+Displacement} \]

Example

```
MOV SI, 1000H
MOV AX, [SI + 20H]
```

Given:

```
SI = 1000H
Displacement = 0020H
```

Step 1:

\[EA=1000H+0020H \]
\[EA=1020H \]

Step 2: CPU accesses memory at DS:1020H.

Suppose:

```
Memory[DS:1020H] = 5678H
```

Result:

```
AX = 5678H
```

Remember: Indexed = Index register + displacement.

16. Based-Indexed Addressing Mode

Definition: In based-indexed addressing mode, the effective memory offset is calculated by adding a base register and an index register.

An optional displacement may also be added.

Formula

\[\boxed{EA=Base+Index+Displacement} \]

Example

```
MOV BX, 1000H
MOV SI, 0200H
MOV AX, [BX + SI]
```

Given:

```
BX = 1000H
SI = 0200H
```

Step 1: Calculate effective address.

\[EA=1000H+0200H \]
\[EA=1200H \]

Step 2: CPU accesses memory at DS:1200H.

Suppose:

```
Memory[DS:1200H] = 5678H
```

Result:

```
AX = 5678H
```

Example with Displacement

```
MOV BX, 1000H
MOV DI, 0200H
MOV AX, [BX + DI + 10H]
```
\[EA=1000H+0200H+0010H \]
\[\boxed{EA=1210H} \]

The CPU accesses memory at DS:1210H.

Remember: Based-indexed = Base + Index + optional displacement.

Addressing modes of 8086 - Binu Joy | PPTX
slideshare.net

17. Complete Comparison of Addressing Modes

Must Prepare for Midterm

Addressing Mode	Where is operand?	Main idea
Immediate	Inside instruction	Constant value
Register	CPU register	Register contains data
Direct	Memory	Address directly provided
Register Indirect	Memory	Register contains address
Based	Memory	Base register + displacement
Indexed	Memory	Index register + displacement

Based-Indexed	Memory	Base + index + displacement

All Addressing Modes with Examples

Mode	Example
Immediate	`MOV AX, 10`
Register	`MOV AX, BX`
Direct	`MOV AX, [2000H]`
Register Indirect	`MOV AX, [BX]`
Based	`MOV AX, [BX+10H]`
Indexed	`MOV AX, [SI+10H]`
Based-Indexed	`MOV AX, [BX+SI]`

18. Important Concept: Effective Address vs Physical Address

This is particularly important because Week 7 and Week 8 concepts are connected.

Effective Address (EA): The offset address calculated by the addressing mode.

Physical Address (PA): The actual memory address calculated by combining the segment address and effective address.

Example

```
MOV BX, 1000H
MOV SI, 0200H
MOV AX, [BX + SI + 10H]
```

Given:

```
DS = 2000H
BX = 1000H
SI = 0200H
Displacement = 0010H
```

Step 1: Calculate effective address.

\[EA=BX+SI+Displacement \]
\[EA=1000H+0200H+0010H \]
\[EA=1210H \]

Step 2: Calculate physical address.

\[PA=(DS\times10H)+EA \]
\[PA=(2000H\times10H)+1210H \]
\[PA=20000H+1210H \]
\[\boxed{PA=21210H} \]

Remember the difference

EA = 1210H

Offset calculated from the addressing mode

PA = 21210H

Actual physical memory address

19. Important 8086 Addressing Rules

These details are useful for MCQs and numerical questions.

Register Used	Default Segment
BX	DS
SI	DS
DI	DS
BP	SS

Important valid 8086 based-indexed combinations:

- BX + SI
- BX + DI
- BP + SI
- BP + DI

A displacement can be added to these combinations.

Examples:

```
MOV AX, [BX + SI]
MOV AX, [BP + DI + 10H]
```

Important: AX, CX, DX, and SP cannot be used as the 8086 base or index registers in these memory-addressing forms.

Important Long Questions for Midterm

Prepare these questions carefully:

1. Define Instruction Set Architecture (ISA) and explain its importance.
2. Differentiate between machine language and assembly language.
3. What is an assembler? Explain its function.
4. Explain opcode and operands with examples.
5. Explain the general instruction format.
6. Explain the different instruction categories with examples.
7. What are addressing modes? Why are they needed?
8. Explain immediate, register, and direct addressing modes with examples.
9. Explain register indirect and based addressing modes.
10. Explain indexed and based-indexed addressing modes with examples.
11. Differentiate between all seven addressing modes.
12. Differentiate between effective address and physical address.

Important Short Questions and Answers

Q1. What is ISA?

ISA is the interface between hardware and software that defines the instructions and rules supported by a processor.

Q2. What is machine language?

Machine language is a binary language consisting of 0s and 1s that is directly understood by the CPU.

Q3. What is assembly language?

Assembly language is a low-level language that uses mnemonics to represent machine instructions.

Q4. What is an assembler?

An assembler converts assembly-language programs into machine code.

Q5. What is an opcode?

An opcode specifies the operation that the CPU must perform.

Q6. What are operands?

Operands are the data or locations on which an instruction performs an operation.

Q7. What is instruction format?

Instruction format defines the arrangement of fields in a machine instruction.

Q8. What are addressing modes?

Addressing modes specify how an instruction obtains its operands.

Q9. What is immediate addressing?

The actual data value is provided directly inside the instruction.

Q10. What is register addressing?

The operand is stored in a CPU register.

Q11. What is direct addressing?

The instruction directly specifies the memory offset of the operand.

Q12. What is register indirect addressing?

A register contains the memory offset of the operand.

Q13. What is based addressing?

The effective address is calculated using a base register and displacement.

Q14. What is indexed addressing?

The effective address is calculated using an index register and displacement.

Q15. What is based-indexed addressing?

The effective address is calculated using a base register, an index register, and optionally a displacement.

Q16. What is effective address?

Effective address is the offset calculated to locate an operand in memory.

Numerical Practice: Addressing Modes

Try these questions yourself. Reveal the solutions after attempting them.

Practice five numerical questions

Q1. BX = 1000H, displacement = 0020H. Find the effective address using based addressing.

Show solution

Q2. SI = 2000H, displacement = 0100H. Find the effective address using indexed addressing.

Show solution

Q3. BX = 1000H, SI = 0200H. Find the effective address using based-indexed addressing.

Show solution

Q4. BX = 1000H, DI = 0200H, displacement = 0010H, DS = 3000H. Find the physical address.

Show solution

Q5. BP = 2000H, SI = 0100H, displacement = 0020H, SS = 4000H. Find the physical address.

Show solution

Week 8 MCQs – Test Yourself

15 MCQs

0/15 answered

1. What does ISA stand for?

Instruction System Application
Instruction Set Architecture
Internal Software Architecture
Integrated System Address

2. Which language is directly understood by the CPU?

Python
C++
Machine language
Assembly language

3. Which program converts assembly language into machine code?

Compiler
Interpreter
Assembler
Debugger

4. Which part of an instruction specifies the operation?

Operand
Opcode
Address
Register

5. In ADD AX, BX, what are the operands?

ADD only
AX and BX
AX only
BX only

6. Which addressing mode provides data directly in the instruction?

Direct
Immediate
Register indirect
Indexed

7. MOV AX, BX is an example of which addressing mode?

Immediate
Direct
Register
Based

8. In MOV AX, [2000H], what does 2000H represent?

Immediate data
A memory offset
An opcode
A register value

9. Which registers are used for register indirect addressing in 8086?

AX, CX, DX, SP
BX, BP, SI, DI
AX, BX, CX, DX
CS, DS, SS, ES

10. Which registers are used for based addressing?

AX and DX
BX and BP
SI and DI
CS and DS

11. Which registers are used for indexed addressing?

AX and BX
SP and BP
SI and DI
CX and DX

12. What is the formula for based-indexed addressing?

Base − Index
Base + Index
Segment × 10H
Opcode + Operand

13. What does EA stand for?

External Address
Effective Address
Execution Architecture
Extended Arithmetic

14. What is the default segment for BP-based memory addressing?

DS
CS
SS
ES

15. Which addressing mode uses a register that holds the memory address?

Immediate
Register
Register indirect
Direct
Submit answers

Final Week 8 Revision Checklist

- [ ] Define ISA and explain its importance.
- [ ] Differentiate machine language and assembly language.
- [ ] Understand assembler.
- [ ] Explain opcode and operands.
- [ ] Understand instruction format.
- [ ] Memorize instruction categories and examples.
- [ ] Explain all seven addressing modes.
- [ ] Differentiate immediate, register, and direct addressing.
- [ ] Understand register indirect addressing.
- [ ] Memorize BX/BP as base registers and SI/DI as index registers.
- [ ] Calculate effective addresses.
- [ ] Calculate physical addresses using segment registers.

My exam recommendation: Give the most attention to the seven addressing modes and their examples. Teachers can easily test these through both theory questions and instruction-based numericals. In particular, practice distinguishing `MOV AX, 2000H` from `MOV AX, [2000H]`, and solving effective-address and physical-address calculations.

You have now covered Weeks 5–8. The next useful step is a combined midterm revision test covering all four weeks, with theory questions, MCQs, and numericals.